

Q10. Hamiltonian Path Problem

Backtracking Approach to an NP-Complete Problem

Field	Details
Student Name	DUDEKULA HUSSAINAIAH
Register Number	192425300
Question No.	Q10
Topic	Constraint-Based Problem Solving – Backtracking

1. Problem Definition

A Hamiltonian Path in a graph $G = (V, E)$ is a path that visits every vertex exactly once — no vertex is repeated and none is skipped. There is no requirement to return to the starting vertex; if it did return, the problem would instead be called a Hamiltonian Cycle.

Formal statement: Given a graph with n vertices, find a permutation of vertices $v_1, v_2, \dots, v_n$ such that for every consecutive pair (v_i, v_{i+1}) , an edge exists in E .

The Hamiltonian Path problem is classified as NP-Complete. No polynomial-time algorithm is known for it, so in the worst case all $n!$ permutations of vertices may need to be examined. Verifying that a given candidate path is valid, however, can be done in polynomial time — which is exactly the property that places the problem in NP. Backtracking is used to prune the search so that the full $n!$ space is rarely explored in practice.

2. Modeling as a Constraint Satisfaction Problem (CSP)

The Hamiltonian Path problem maps naturally onto the standard CSP framework of variables, domains, and constraints:

CSP Element	Hamiltonian Path Mapping
Variables	Position in the path: $pos_1, pos_2, \dots, pos_n$
Domain	Each position may take any vertex from V
Constraints	(1) All-different — no vertex repeats. (2) Adjacency — consecutive vertices must be connected by an edge.

2.1 Constraints in Detail

- Vertex Visitation Constraint: every vertex must appear exactly once in the path (a bijection between positions and vertices, enforced with a `visited[]` array).
- Path Continuity Constraint: for any two consecutive vertices in the path, (current, next) must be an edge of the graph — the path can never “jump” to a non-adjacent vertex.
- Starting Constraint (optional variant): some problem statements fix a starting vertex; the general search instead tries every vertex as a possible start.

2.2 How These Constraints Shape the Solution Space

- Without constraints, the raw search space is all $n!$ permutations of the vertex set — this grows explosively even for modest n (e.g. $10! = 3,628,800$).
- The adjacency constraint prunes this space sharply: at each step only unvisited neighbours of the current vertex are considered, not all remaining vertices. Sparse graphs are pruned aggressively; dense/complete graphs are pruned very little because almost every vertex is a neighbour.
- The all-different constraint is enforced with a running `visited[]` record of the partial assignment, guaranteeing no vertex is chosen twice.
- Net effect: backtracking converts brute-force permutation generation into an early-terminating depth-first search — invalid branches are abandoned the moment a constraint is violated, rather than being completed and checked afterward.

3. Backtracking Approach

The path is built incrementally, one vertex at a time. At each step the algorithm attempts to extend the current path with an unvisited vertex that is adjacent to the last vertex placed. If a dead end is reached — no valid vertex can extend the path — the algorithm backtracks: it undoes the most recent choice and tries the next alternative.

3.1 Algorithm Steps

1. Start with an empty path; pick a starting vertex (or try each vertex as a start in turn).
2. Mark the starting vertex visited and add it to the path.
3. Recursively attempt to add an unvisited neighbour of the last vertex in the path.
4. Constraint check before adding: the candidate vertex must be (a) adjacent to the current last vertex, and (b) not already visited.
5. If the path includes all n vertices, a solution has been found.
6. If no neighbour can be validly added, backtrack: remove the last vertex, mark it unvisited again, and try the next candidate.
7. If all options at the root are exhausted with no success, no Hamiltonian Path exists (from that start, or at all).

4. Python Implementation

The implementation below directly reflects the CSP model: `is_valid()` performs the constraint checking (continuity + visitation), and `_backtrack()` performs the depth-first search with undo-on-failure.

```
class HamiltonianPath:
    def __init__(self, graph, n):
        self.graph = graph      # adjacency list
        self.n = n              # number of vertices
        self.path = [-1] * n
        self.visited = [False] * n

    def is_valid(self, vertex, pos):
        # Constraint 1: Path Continuity
        if pos > 0:
            prev = self.path[pos - 1]
            if vertex not in self.graph[prev]:
                return False
        # Constraint 2: Vertex Visitation (all-different)
        if self.visited[vertex]:
            return False
        return True

    def solve_from(self, start):
        self.path = [-1] * self.n
        self.visited = [False] * self.n
        self.path[0] = start
        self.visited[start] = True
        if self._backtrack(1):
            return list(self.path)
        return None

    def _backtrack(self, pos):
        if pos == self.n:      # all vertices placed
            return True
        for vertex in range(self.n):
            if self.is_valid(vertex, pos):
                self.path[pos] = vertex
                self.visited[vertex] = True
                if self._backtrack(pos + 1):
                    return True
                self.path[pos] = -1    # backtrack: undo
                self.visited[vertex] = False
        return False

    def find_any_hamiltonian_path(self):
        for start in range(self.n):
            result = self.solve_from(start)
            if result:
```

```
    return result
return None
```

Sample run:

```
graph = {0: [1, 2], 1: [0, 2, 3], 2: [0, 1, 4], 3: [1, 4], 4: [2, 3]}
>>> HamiltonianPath(graph, 5).find_any_hamiltonian_path()
Hamiltonian Path exists: 0 -> 1 -> 2 -> 4 -> 3

graph2 = {0: [1, 2, 3], 1: [0], 2: [0], 3: [0]} # star graph
>>> HamiltonianPath(graph2, 4).find_any_hamiltonian_path()
No Hamiltonian Path exists.
```

5. Trace Example

Graph edges: 0–1, 0–2, 1–2, 1–3, 2–4, 3–4. Starting vertex: 0.

Step	Path so far	Constraint Check / Action
1	[0]	Start at vertex 0
2	[0, 1]	1 is adjacent to 0 and unvisited → valid
3	[0, 1, 2]	2 is adjacent to 1 and unvisited → valid
4	[0, 1, 2, 4]	4 is adjacent to 2 and unvisited → valid
5	[0, 1, 2, 4, 3]	3 is adjacent to 4 and unvisited → all 5 vertices placed: solution found

If, at any step, no valid unvisited neighbour exists for the current vertex, the algorithm backtracks to the previous position and tries a different candidate there. This undo-and-retry step is the core pruning mechanism that avoids exhaustively generating and checking all $n!$ permutations.

6. Complexity Analysis

Aspect	Complexity / Note
Time (worst case)	$O(n!)$ — occurs on dense/complete graphs, where backtracking prunes very little
Time (with pruning)	Substantially better on sparse graphs, since the adjacency constraint eliminates most branches early
Space	$O(n)$ for the path array plus $O(n)$ recursion stack depth
Problem class	NP-Complete — verifying a given path takes $O(n)$, but finding one is exponential in the worst case

7. Summary

- The Hamiltonian Path problem is modeled as a CSP with an all-different constraint (no repeated vertices) and an adjacency / continuity constraint (consecutive path vertices must share an edge).
- Backtracking explores the search tree depth-first, applying the `is_valid()` constraint check before every assignment — this is constraint checking put directly into practice.
- When a constraint fails, the algorithm backtracks instead of completing and then rejecting a full permutation, which meaningfully reduces the effective search space compared to brute-force generation — even though the worst-case bound remains exponential, consistent with the problem's NP-Complete status.